

Encodage one-hot des séquences

Décomposition de `np.eye(A, dtype=np.float32)[sequences]`

Institut de la Vision — Directed Evolution Loop

1 Le code

```
sequences_arr = np.asarray(sequences)
N_seqs       = sequences_arr.shape[0]
X             = np.eye(A, dtype=np.float32)[sequences] # shape (N, L, A)
return X.reshape(N_seqs, L*A)
```

Ce code transforme une librairie de N séquences de longueur L (chacune une suite d'indices d'acides aminés dans $\{0, \dots, A-1\}$) en une matrice de caractéristiques *one-hot* de taille $(N, L \cdot A)$, prête à être utilisée comme matrice de design X dans `build_potts_features`.

2 Étape par étape

Étape 1 : `np.asarray(sequences)`

Convertit `sequences` (qui peut être une liste, un tableau JAX, etc.) en un tableau NumPy standard, de forme (N, L) , à valeurs entières dans $\{0, \dots, A-1\}$. C'est purement une conversion de type ; aucune donnée n'est modifiée.

Étape 2 : `np.eye(A, dtype=np.float32)`

Construit la matrice identité $A \times A$:

$$I_A = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \in \mathbb{R}^{A \times A}.$$

La ligne k de I_A est le vecteur one-hot de l'acide aminé k : que des zéros, sauf un 1 en position k .

Étape 3 : `indexation [sequences]`

C'est l'étape la moins évidente. NumPy interprète `sequences` (un tableau d'entiers de forme (N, L)) comme un tableau d'*indices de lignes* à extraire de I_A . Pour chaque entrée `sequences[n, l] = k`, NumPy va chercher la ligne k de I_A et la place à la position (n, l) du résultat. Le résultat a donc une dimension de plus que l'entrée :

$$\underbrace{I_A}_{(A,A)} \left[\underbrace{\text{sequences}}_{(N,L)} \right] \longrightarrow \underbrace{X}_{(N,L,A)} \quad \text{avec} \quad X[n, l, :] = I_A[\text{sequences}[n, l], :].$$

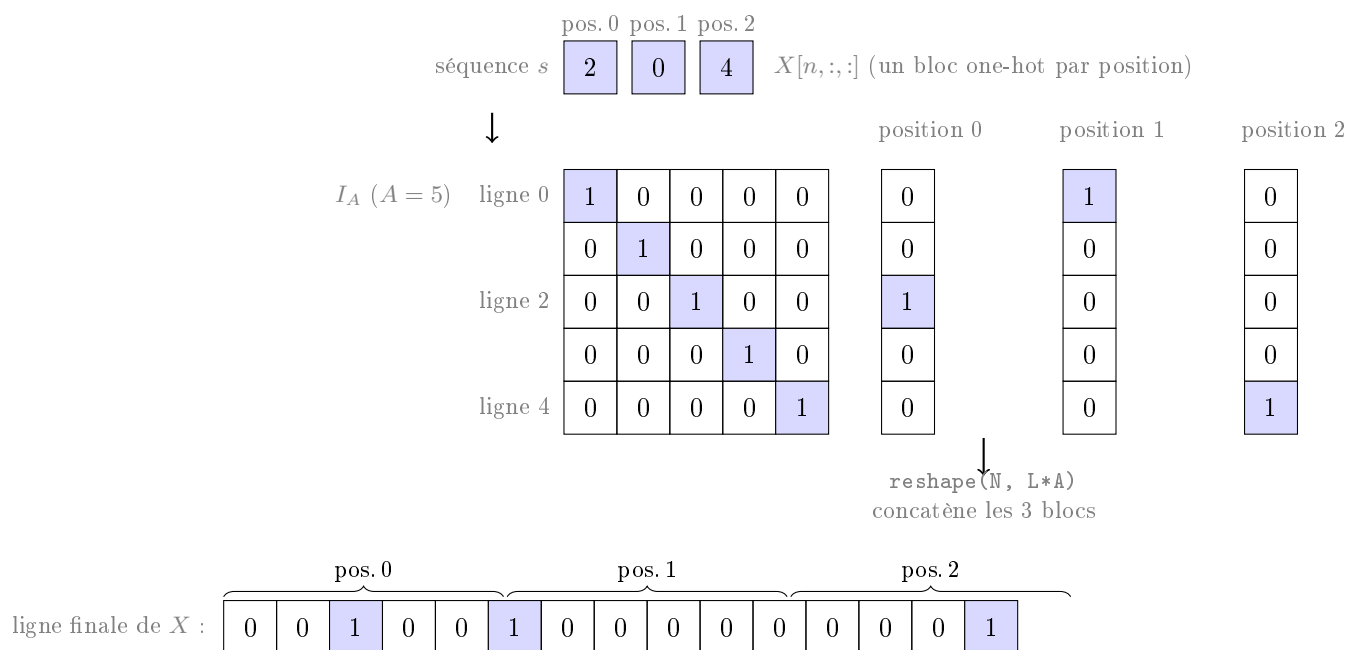
Autrement dit : pour chaque séquence n et chaque position l , on obtient le vecteur one-hot (de longueur A) de l'acide aminé qui s'y trouve.

Étape 4 : `X.reshape(N_seqs, L*A)`

On aplatit les deux dernières dimensions (L, A) en une seule de taille $L \cdot A$, en concaténant les L blocs one-hot les uns à la suite des autres. Chaque ligne de la matrice finale est donc un vecteur creux de longueur $L \cdot A$, avec exactement L entrées à 1 (une par position) et le reste à 0.

3 Schéma

L'exemple ci-dessous utilise $L = 3$ positions et $A = 5$ acides aminés (au lieu de $L = 7, A = 20$ dans le code réel) pour que le schéma tienne sur la page. La séquence $s = (2, 0, 4)$ signifie : acide aminé 2 en position 0, acide aminé 0 en position 1, acide aminé 4 en position 2.



Les cases bleutées marquent les 1 (une seule par position) ; toutes les autres cases valent 0. La ligne finale contient exactement $L = 3$ uns sur $L \cdot A = 15$ entrées — un vecteur **creux** (*sparse*) : dans le code réel, $L = 7$ et $A = 20$, donc chaque ligne ne compte que 7 entrées non nulles sur 140 (et, une fois les termes de paires ajoutés, $7 + 21 + 1 = 29$ entrées non nulles sur 8541).

4 Résumé

`np.eye(A)[sequences]` exploite l'indexation *fancy indexing* de NumPy : indexer la matrice identité par un tableau d'entiers revient à sélectionner, pour chaque entier, la ligne correspondante de l'identité — c'est-à-dire son vecteur one-hot. Appliqué à un tableau de séquences (N, L) , cela produit directement un tenseur (N, L, A) de vecteurs one-hot, un par position et par séquence, sans boucle Python explicite. Le **reshape** final aplatit ces L blocs en une seule ligne de taille $L \cdot A$ par séquence — exactement la représentation attendue par la régression Ridge sur les caractéristiques de Potts.